TestNG

Overview:

- It is an open source **testing framework used to design, organize, and execute automated tests written in Java**.
- TestNG stands for "**Test Next Generation**"
- It was created by taking inspiration from **JUnit.**
    - JUnit works well for **unit testing by developers**, but automation testers often need more advanced features like:
        - Running tests in parallel
        - Running tests based on dependencies
        - Passing multiple datasets
        - Running tests from configuration files
- It supports annotations. Annotations are special markers placed above methods to define **how and when they should run** during test execution.
- Factory in TestNG allows **dynamic creation of test instances at runtime**.
    - This is useful when tests must run with **different environments or configurations**.
- `testng.xml` is the **main configuration file used to control test execution**.

Features:

- Annotations in TestNG are simple to understand and provide powerful control over test execution.
- It supports running multiple tests simultaneously through parallel execution.
- Test cases can be organized using groups, priorities, and execution order.
- Generates comprehensive test reports and execution logs.
- Enables data-driven testing using the DataProvider feature.
- Easily integrates with build and automation tools such as Maven, Jenkins, and CI/CD pipelines.

Execution Flow of TestNG Annotations:

TestNG provides several annotations that control **how and when different parts of a test suite execute**. These annotations help testers organize setup activities, test execution, and cleanup operations in a structured way.

Overall Execution order:

@BeforeSuite
@BeforeTest
@BeforeClass
@BeforeMethod
@Test
@AfterMethod
@AfterClass
@AfterTest
@AfterSuite

- **@BeforeSuite:**
  - It executes **once before the entire test suite starts**. It is typically used for **global setup activities** that are required for all tests in the suite.
  - Example: Before any tests run, the framework may need to: Load configuration files (environment URLs, browser settings), Initialize reporting tools, Establish database connections if required.
- **@BeforeTest:**
  - It runs **before the execution of any <test> tag defined in the TestNG XML file**.It is commonly used for **test-level setup**.
  - Example: Assume your testng.xml file contains separate tests for: Login functionality, Product search, Cart and checkout. Before executing the **Login test group**, you may want to: Launch the browser, Load test-specific configuration, Set browser preferences.
- **@BeforeClass:**
  - It runs **once before the first test method in a specific test class**. It is useful for **class-level initialization**.
  - It will be executed only once in a entire class execution.
  - Example: Before executing any login-related tests, you may need to: Initialize the WebDriver instance, Create Page Object instances, Prepare test data
- **@BeforeMethod:**
  - It executes **before each individual @Test method** in a class. This ensures that every test starts with a **fresh and consistent state**.
  - It will not execute without any @Test methods, as they are configuration methods. Use it to handle common preconditions like login for all test cases.
  - Example: Consider a login test class containing multiple test cases: Valid login, Invalid password, Empty credentials. Before each test runs, the framework may: Navigate to the login page, Reset the application state, Ensure the user is logged out

- **@Test:**
  - TestNG execution starts from methods annotated with @Test.
  - Without @Test, the class will not execute; it acts like an entry point similar to the main method.
  - A TestNG class can contain multiple test methods, but each must be annotated with @Test.
  - Test methods should be public with void return type; method names can be anything.
  - As a best practice, the class name should represent the module, and method names should match manual test case names.
  - Class names and test method names should end with "**Test**" as per naming conventions.
  - It represents the **actual test case** where the automation logic is implemented. This is where testers write the **business workflow they want to validate**.
  - Example: An E-commerce app may include, Verifying successful login with valid credentials, Searching for a product, Adding a product to the cart, Completing the checkout process
- **@AfterMethod:**
  - It runs **after each @Test method completes**.It is used for **cleanup or post-test actions**.
  - It will not execute without any @Test methods, as they are configuration methods. Use it to handle common postconditions like logout for all test cases.
  - Example: After each test, the framework may need to: Log out the user, Clear browser cookies, Capture screenshots if the test fails
- **@AfterClass:**
  - It executes **once after all test methods in a class have finished**. This annotation is typically used for **class-level cleanup**.
  - It will be executed only once in a entire class execution.
  - Example: After all login-related tests are completed, the framework may: Close the browser, Release resources associated with that test class
- **@AfterTest:**
  - It runs **after all <test> tags in the TestNG XML file have finished executing**. This is useful for **test-level cleanup activities**.
  - Example: After completing all tests related to a specific module (for example, Cart functionality), the framework may: Clear test data, Close remaining browser sessions, Reset environment configurations
- **@AfterSuite:**
  - It runs **once after the entire test suite execution has completed**. It is mainly used for **final reporting and global cleanup**.

- Example: Once the entire regression suite finishes, the framework may: Generate execution reports, Send email notifications with test results, Close database connections

---

Parallel test execution:

- TestNG provides the ability to **run multiple test cases simultaneously instead of sequentially**. This is mainly controlled through the **testng.xml configuration file**, where testers define how tests should execute.
  - Example: In a real automation framework, teams often maintain **large regression suites**. Running tests one after another can take a long time. Parallel execution helps: Reduce execution time, Run tests across multiple browsers, Speed up CI/CD pipelines
- TestNG allows parallel execution at **different levels of test execution** like methods, classes, or test suites.
  - Example: Different projects choose different levels depending on their framework design. Examples: **methods** → useful when test methods are independent, **classes** → common for Page Object Model frameworks, **tests** → used for cross-browser testing
- In `testng.xml`, the `parallel` attribute defines **how TestNG distributes test execution across threads**.
  - Example values: parallel="methods", parallel="classes", parallel="tests"
- Sequential execution runs tests one after another in a defined order, making it suitable for scenarios where tests depend on each other. In contrast, parallel execution runs multiple tests simultaneously, which is ideal for independent tests and helps reduce overall execution time.
- Thread Pooling means: Instead of creating unlimited threads, TestNG uses a **fixed number of threads (thread-count).** These threads are reused to execute multiple test methods.
  - `thread-count` defines **how many tests can run simultaneously**.
  - Example: thread-count="3", This means **3 tests can run in parallel at the same time**.
  - Thread count usually depends on: Machine capacity, Browser instances, Infrastructure resources. For example: Local machine → 2–4 threads, CI pipeline → 10–20 threads
  - Thread pooling helps: Prevent system overload, Control CPU & memory usage, Maintain stable execution
- Parallel execution improves: Speed, Resource utilization, Faster feedback

Practical Examples:

- **Parallel Execution by Methods:** It allows individual test methods to run concurrently.

Scenario: We want to verify **multiple login scenarios simultaneously** for an E-commerce app.

```
public class LoginTests {

    @Test

    public void verifyValidLogin() {

        System.out.println("Valid Login Test - " +
Thread.currentThread().getId());

    }

    @Test

    public void verifyInvalidLogin() {

        System.out.println("Invalid Login Test - " +
Thread.currentThread().getId());

    }

    @Test

    public void verifyEmptyCredentials() {

        System.out.println("Empty Login Test - " +
Thread.currentThread().getId());

    }
```

testng.xml Configuration:

```
<suite name="AutomationSuite" parallel="methods" thread-count="3">
    <test name="LoginTests">
        <classes>
            <class name="tests.LoginTests"/>
        </classes>
    </test>
</suite>
```

All three login tests run simultaneously in separate threads.

```
Example console output:
Valid Login Test - Thread 12
Invalid Login Test - Thread 13
Empty Login Test - Thread 14
```

This proves the tests ran in parallel.

- **Parallel Execution by Classes:** All test methods within a class are executed in parallel.

Scenario: We want to run different modules simultaneously: Login tests, Product tests, Cart tests

testng.xml:

```
<suite name="AutomationSuite" parallel="classes" thread-count="3">
    <test name="EcommerceTests">
        <classes>
            <class name="tests.LoginTests"/>
            <class name="tests.ProductTests"/>
            <class name="tests.CartTests"/>
        </classes>
    </test>
</suite>
```

TestNG runs: LoginTests, ProductTests, CartTests **at the same time**, each in a separate thread. This significantly speeds up **regression execution**.

- **Parallel Execution by Tests:** Entire test sections defined in testng.xml run simultaneously.

Scenario: We want to run tests on **multiple browsers simultaneously**.

testng.xml:

```
<suite name="CrossBrowserSuite" parallel="tests" thread-count="2">
    <test name="ChromeTests">
        <parameter name="browser" value="chrome"/>
        <classes>
            <class name="tests.LoginTests"/>
        </classes>
    </test>
    <test name="FirefoxTests">
        <parameter name="browser" value="firefox"/>
        <classes>
            <class name="tests.LoginTests"/>
```

```
        </classes>
    </test>
</suite>
```

The same **login test** runs simultaneously on: Chrome and Firefox. This is commonly used for **cross-browser testing in the CI pipeline.**

- For large test suites, Instead of running all tests together, divide them into logical groups like smoke, regression, sanity.
    - Use thread-safe classes: Each thread should have its **own instance** of objects (especially WebDriver). Multiple threads using same driver leads to conflicts.

      ```
      public class DriverManager {

          private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

          public static void setDriver(WebDriver drv) {

              driver.set(drv);

          }

          public static WebDriver getDriver() {

              return driver.get();

          }

      }
      ```

    - Avoid shared data where possible
        - Do not share: Static variables, Global test data
- Prevent deadlocks during parallel execution:
    - Avoid shared resources like same file, same DB record, same WebDriver
    - Provide separate resources per test: Unique users, Separate browsers, Independent test data

---

Configuring TestNG in a Project and the Role of `testng.xml`:

- Once TestNG is added to a project (through Maven dependency or IDE integration), test execution is typically managed using a **`testng.xml` configuration file**. This file acts as the **central control point** for organizing and running test suites.
- Using `testng.xml`, testers can:

    - Define **test suites and test modules**
    - Specify which **test classes or methods should run**
    - Include or exclude certain tests
    - Configure **parallel execution**
    - Passing **parameters** to test methods
    - **Grouping test cases** into logical modules
    - Control the order of test execution
- Test suites can then be executed directly from **IDE tools such as Eclipse or IntelliJ**, or automatically through **build tools like Maven** in CI/CD pipelines.

@BeforeGroups:

- It is executed **once before the first test method of a specific group is run**.

@AfterGroups:

- It is executed **once after all test methods of that group have finished execution**.

Example:

```
public class GroupExample {

//Before executing login tests: Open login page, Ensure no user is logged in
@BeforeGroups("loginTests")
public void setupLoginGroup() {
System.out.println("Before Login Group - Open Login Page");
}

//Runs after all loginTests group methods complete. Close login session, Reset login state
@AfterGroups("loginTests")
public void tearDownLoginGroup() {
System.out.println("After Login Group - Close Login Session");
}

//Runs before the first cart-related test executes. Login user (precondition) Navigate to product page
@BeforeGroups("cartTests")
public void setupCartGroup() {
System.out.println("Before Cart Group - Login User");
}
```

```
//Runs after all cart tests complete. Clear cart items, Logout user
@AfterGroups("cartTests")
public void tearDownCartGroup() {
System.out.println("After Cart Group - Clear Cart Data");
}

//This test belongs to the "loginTests" group. Valid login test. @Test(groups
= "loginTests")
public void verifyValidLogin() {
System.out.println("Executing Valid Login Test");
}

//This test belongs to the "loginTests" group. Invalid login test
@Test(groups = "loginTests")
public void verifyInvalidLogin() {
System.out.println("Executing Invalid Login Test");
}

//This test belongs to "cartTests" group. Add product to cart. Validate cart
items
@Test(groups = "cartTests")
public void verifyAddToCart() {
System.out.println("Executing Add to Cart Test");
} }
```

@DataProvider:

- It allows a single test method to run **multiple times with different input values (parameterized testing)** instead of writing separate test cases.
- A method annotated with `@DataProvider` returns data (usually in a 2D Object array), and this data is **passed to the test method as parameters**.
- The test method is executed **once for each data set provided**. Instead of duplicating code, we reuse the same logic with different inputs.
- TestNG supports using data from external sources for data-driven testing. You can read data from files such as Excel, CSV, XML, or JSON using libraries like Apache POI (for Excel), and then supply that data to test methods through the `@DataProvider` annotation.

Example:

```
public class TestData {
/*Defines a data provider named "loginData". This name is used to link the
data with the test method. */
```

```
@DataProvider(name = "loginData")
public Object[][] getLoginData() {
return new Object[][] {
{"testuser", "correctpassword"},
{"testuser", "wrongpassword"},
{"invaliduser", "test123"}
}; } }

public class LoginTests {

//Links the test method with the DataProvider.
@Test(dataProvider = "loginData", dataProviderClass = TestData.class)
public void verifyLogin(String username, String password) {

System.out.println("Executing test with: " + username + " | " + passwor d);

// Example real steps (conceptual)
// 1. Open Automation Test Store login page
// 2. Enter username
// 3. Enter password
// 4. Click login
// 5. Validate result } }
```

- We can also integrate database with testng.
  - To fetch data from a database, we first need to **connect to the database** using: JDBC (Java Database Connectivity), Database URL, Username & Password
  - After connecting, we execute **SQL queries** to fetch required data.
  - After fetching data, we convert it into a format usable by TestNG (usually `Object[][]`) and pass it via `@DataProvider`.

Example:

```
public class DBDataProvider {

@DataProvider(name = "dbData")
public Object[][] getDataFromDB() throws Exception {

Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/testdb", "user", "password");

Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT username, password FROM users");

List<Object[]> data = new ArrayList<>();
```

```
while (rs.next()) {
data.add(new Object[] {
rs.getString("username"),
rs.getString("password")
});
}

return data.toArray(new Object[0][]);
}
}
```

Test Class Using DB Data:

```
public class LoginTest {

   @Test(dataProvider = "dbData", dataProviderClass = DBDataProvider.class)
   public void verifyLogin(String username, String password) {

       System.out.println("Testing with: " + username + " | " + password);

       // Real scenario:
       // 1. Open Automation Test Store
       // 2. Enter username & password
       // 3. Click login
       // 4. Validate result
   }
}
```

Example Use cases:

- Login Testing(Test multiple credential combinations)
- Form Validation (Test Empty fields, Invalid formats, Boundary values)
- Search Functionality (Test multiple search keywords: Valid product, Invalid product, Special characters)
- Checkout Testing (Test Different payment methods, Different shipping details)

@Factory:

- It is used to **dynamically generate test class instances at runtime**, instead of creating them manually.
- Technically, `@Factory` creates **multiple instances of a test class**, and each instance executes its test methods.

- Each instance of the test class is created with **different constructor parameters**, and those parameters control how the test behaves.
- Instead of passing data to methods (@DataProvider), here we pass data to the **entire test class instance**.

Example:

```
public class LoginTest {
   String browser;
   // Constructor. Each test instance will receive a browser value.
   public LoginTest(String browser) {
       this.browser = browser; //Stores browser value for use in test
execution.
   }
   // Same test method runs, but behavior changes based on the browser.
   @Test
   public void verifyLogin() {
       System.out.println("Executing Login Test on: " + browser);

       // Real scenario steps:
       // 1. Launch browser (based on browser variable)
       // 2. Open Automation Test Store
       // 3. Perform login
       // 4. Validate result
   }
}

public class TestFactory {
// @Factory Marks a method that returns multiple test class instances.
@Factory
public Object[] createTests() {

return new Object[] {
new LoginTest("Chrome"),
new LoginTest("Firefox"),
new LoginTest("Edge")
};  }  }
```

Real Use Cases:

- Cross-Browser Testing (Run same tests on: Chrome, Firefox, Edge)
- Multi-User Testing (Test with: Admin user, Normal user, Guest user)
- Environment Testing (Run same tests on: QA environment, Staging, Production)

Feature	@DataProvider	@Factory
Works on	Test method	Test class
Use case	Input data	Environment/config
Execution	Same instance	Multiple instances

@Listeners:

- Listeners in TestNG are interfaces that monitor test execution events. It is used to **attach custom logic to the test execution lifecycle** by implementing TestNG listener interfaces(ITestListener, ISuiteListener, etc.).
- It allows you to **intercept events** like: Test start, Test success, Test failure, Suite start/end.
- Listeners provide predefined methods that are triggered automatically when certain events occur.
- Listeners help add **extra functionality** without modifying test cases. When test starts → log test name, When test fails → capture screenshot, When test completes → update report
- ITestListener: captures events during test execution such as test start, success, failure, and finish.
    -

Test starts	onTestStart()
Test passes	onTestSuccess()
Test fails	onTestFailure()
Test end	onFinish()

- IReporter: It is used to create **custom reports** beyond default TestNG reports.
    - Default reports are basic. But in real projects, we need: Custom HTML reports, Dashboard view, Graphs and summaries
- Instead of using `System.out.println`, we use professional logging tools like: Log4j, SLF4J. Logging can be added: Inside test methods, Inside listeners (recommended).
- End to End flow:

  Test Starts → Listener logs start

  Test Executes

  If PASS → log success

  If FAIL → capture screenshot + log error

  Report Generated → includes logs + screenshots

Example:

```
    public class TestListener implements ITestListener {

    @Override
    public void onTestStart(ITestResult result) {
    System.out.println("Test Started: " + result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {
    System.out.println("Test Passed: " + result.getName());
    }

    @Override
    public void onTestFailure(ITestResult result) {
    System.out.println("Test Failed: " + result.getName());

    // Real-world example:
    // Capture screenshot here
    // takeScreenshot(result.getName());
    }
    }

    @Listeners(TestListener.class) //Links the listener with the test class.
    public class LoginTests {
    @Test
    public void verifyLogin() {
    System.out.println("Executing Login Test");

    // Simulate failure
    assert false;
    }
    @Test
    public void verifyAddToCart() {
    System.out.println("Executing Add to Cart Test");
    }
    }
```

Use Cases: Screenshot Capture on Failure, Reporting Integration(Extent reports, Allure Reports), Logging Framework Integration(Log4j, SLF4J), Retry Mechanism

Advantages:

- Listener can be used for **setup actions** like Connect to DB, launch browser, perform login.
- Automatically capture screenshot when test fails.

Assertions:

- Assertions are used to **verify whether the actual result matches the expected result**.
- Assertions act as **checkpoints** in your test.
- If expected ≠ actual → TestNG marks test as **FAILED**
- Assertions decide the **final status of the test case**. Match → PASS, Mismatch → FAIL
- After execution, TestNG generates **detailed information about each test**. It includes(Test status (PASS / FAIL), Failure reason, Execution time, Logs)
  Example: Test Started: verifyLogin
  Executing Login Test
  Test Failed: verifyLogin
  java.lang.AssertionError: expected [My Account] but found [Login]
- TestNG generates reports in multiple formats:
  Console logs - Quick debugging
  HTML report - Visual analysis
  XML report - CI/CD integration

Most frequently used validation methods:

assertEquals:

- It is used to check if two values are equal.

Example:

```
public class LoginTest {

@Test
public void verifyLoginSuccess() {
String expectedTitle = "My Account";
String actualTitle = "My Account"; // assume fetched from UI

Assert.assertEquals(actualTitle, expectedTitle);
}
}
```

assertTrue:

- It is used to check that a condition is true.

Example: Verify login button is displayed

```
Assert.assertTrue(isLoginButtonDisplayed);
```

assertFalse:

- It used to check that a condition is false

Example: Verify error message is NOT displayed

```
Assert.assertFalse(isErrorMessageDisplayed);
```

assertNull:

- It checks if an object is null.

Example: Before login:

```
Assert.assertNull(userSession); //No user session should exist
```

assertNotNull:

- It checks if an object is not null.

Example: After login:

```
Assert.assertNotNull(userSession); //Session should be created
```

Hard Assertions:

- It stops the test execution if the assertion fails.
- Real Scenario: If login fails → no need to check cart → stop test
- All methods are static.
- Use **hard assertions** when the validation is **critical** for the test to proceed.
    - If this fails → test should stop immediately.

Example:

```
Assert.assertEquals("Home", "Login"); // FAIL
System.out.println("This will NOT execute");
```

Soft Assertions:

- It allows the test to continue executing even if the assertion fails.
- The test result is evaluated at the end by calling assertAll().
- Real Scenario: verify logo, search bar, menu, footer. Even if logo fails → we still want to check others

- All methods are non-static.
- Use **soft assertions** when validations are **not critical**, and you want the test to continue even if they fail.

Example:

```
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals("Home", "Login"); // FAIL
System.out.println("This WILL execute");
softAssert.assertAll();
```

Prioritize test cases in TestNG:

- TestNG allows you to control the **execution order of test methods** by assigning a priority value. Ex. @Test(priority = 1). We can pass both positive and negative value.
- It executes test methods in **ascending order of priority**.
- If no priority is specified, the test methods are executed alphabetically by method name.
- When multiple test methods share the same priority, TestNG cannot decide order based on priority. In such cases, TestNG falls back to **alphabetical sorting**.\
- Avoid heavy reliance on priority
- Use priority only for simple sequencing.
- If no priority assigned then default priority is 0.

Problem Scenario:
If all tests have same priority:

- Execution becomes unpredictable for business flow
- Tests may fail due to dependency issues

Group test cases in TestNG:

- TestNG allows you to assign **logical categories (groups)** to test methods using:the groups attribute in the @Test annotation.
  Ex. @Test(groups = "groupName")

- A single test method can be part of **multiple logical categories**. Grouping execution means running a collection of related test scripts across different classes. To achieve this, each test method must be assigned a **group name** using the @Test annotation. For configuration methods (@BeforeSuite, @BeforeClass,

@BeforeMethod, etc.), the same group name should be defined; otherwise, they won't be included in group execution.

Ex. @Test(groups = {"smokeTests", "loginTests"})
public void verifyLogin() {
    System.out.println("Login Test");
}

- Multiple group keys can be defined in a single XML file.A single test method can belong to multiple groups.
- To execute specific groups, define the **group key in testng.xml**, placed **after <suite> and before <test> tag**. Once groups are defined, you can run: Specific groups, Multiple groups together using testng.xml.
Ex.<suite name="AutomationSuite">

```
    <test name="LoginTests">
        <groups>
            <run>
                <include name="loginTests"/>
            </run>
        </groups>

        <classes>
            <class name="tests.GroupExample"/>
        </classes>
    </test>

</suite>
```

- Include / Exclude Groups: TestNG allows selective execution of tests based on groups. You can define which groups to include or exclude using the <groups> tag and <include> or <exclude> elements.
- Advantages of grouping:
  - Allows running only specific sets of tests based on group names.
  - Helps structure large test suites by categorizing tests into logical groups.
  - Enables quick execution of only relevant or failing groups during debugging.

Dependency Management:

- TestNG allows you to define that **one test method depends on another test method**. The dependsOnMethods attribute is used to define dependencies between test methods. A test method will execute only if the method it depends on

passes. If the dependent method fails or is skipped, the dependent test will not be executed.
Ex. @Test(dependsOnMethods = "methodName")

```
public class DependencyTest {

@Test
public void login() {
System.out.println("Login Successful");
Assert.assertTrue(true);
}

@Test(dependsOnMethods = "login")
public void addToCart() {
System.out.println("Product added to cart");
}
}
```

- Dependent test runs **only if parent test passes**.
- Instead of depending on a single method, a test can depend on **a group of tests**.dependsOnGroups attribute is used for that.

```
public class GroupDependencyTest {

@Test(groups = "loginTests")
public void verifyLogin() {
System.out.println("Login Test");
Assert.assertTrue(true);
}

@Test(groups = "loginTests")
public void verifyInvalidLogin() {
System.out.println("Invalid Login Test");
Assert.assertTrue(true);
}

@Test(dependsOnGroups = "loginTests")
public void addToCart() {
System.out.println("Add to Cart Test");
}
}
```

- All tests in the group must **PASS** for dependent test to run.
- If the parent test fails, dependency chain breaks. TestNG does NOT execute dependent test. Test status becomes **SKIPPED**, not FAILED.

```
public class FailureDependencyTest {

@Test
public void login() {
System.out.println("Login Failed");
Assert.assertTrue(false); // force failure
}

@Test(dependsOnMethods = "login")
public void addToCart() {
System.out.println("Add to Cart");
}
}

Execution Result:
login → FAILED
addToCart → SKIPPED
```

Parameters:

- `@Parameters` allows you to **pass external values to test methods** instead of hardcoding them. Ex. @Parameters("paramName")
- All parameter values are defined inside **testng.xml**, not inside the code.

```
<suite name="AutomationSuite">

    <test name="LoginTest">

        <parameter name="browser" value="chrome"/>
        <parameter name="url" value="https://automationteststore.com"/>

        <classes>
            <class name="tests.ParameterTest"/>
        </classes>

    </test>

</suite>
```

- TestNG automatically passes values from XML into the test method when execution starts.

```
public class ParameterTest {
```

```
@Test
@Parameters({"browser", "url"})
public void launchApp(String browser, String url) {
System.out.println("Browser: " + browser);
System.out.println("URL: " + url);
// Real scenario:
// 1. Launch browser
// 2. Open Automation Test Store URL
}
}
```

- `@Parameters` works with **simple data types only**: String, int, boolean

---

Skipping tests:

- TestNG allows you to **disable a test method** using: `enabled` attribute of the @Test annotation to false. Ex. @Test(enabled = false)
- In real projects, we may need to skip tests when: Feature is under development, Bug is not fixed yet, Test is unstable (flaky), Environment issue.
- Even though the test is not executed: TestNG still tracks it, Marks it as **SKIPPED** in reports.
- Disabled tests are still part of the test suite, so they appear in reports.

```
public class SkipTestExample {

@Test
public void verifyLogin() {
System.out.println("Login Test Executed");
}

@Test(enabled = false)
public void verifyCheckout() {
System.out.println("Checkout Test Executed");
```

```
}
}
```

In reports:

- Login → PASS
- Checkout → SKIPPED

Different ways to skip tests in TestNG:

- SkipException: Allows skipping tests dynamically during execution based on specific conditions (e.g., environment issues or failed preconditions).
  - Ex. throw new SkipException("Skipping test1");
- @Test(enabled = false):
  - Disables a test at the configuration level, ensuring it is not executed at all during the test run.
- Conditional Logic in Test Method:
  - Tests can be skipped programmatically by adding custom conditions within the test logic.
- dependsOnMethods:
  - Automatically skips dependent tests if the methods they rely on fail or are skipped.
- Using Groups:
  - Specific groups of tests can be excluded in the testng.xml file to prevent their execution.

Timeout:

- timeout test annotation is used to specify the maximum amount of time a test method can take to execute. Ex. @Test(timeout = 5000) // in milliseconds
- If execution time > defined timeout: Test is marked as **FAILED.**
- Timeout value is always in **milliseconds.**

```
public class TimeoutTest {

@Test(timeout = 5000)

public void verifyPageLoad() throws InterruptedException {

System.out.println("Opening Automation Test Store...");

Thread.sleep(3000); // simulate page load

System.out.println("Page Loaded Successfully");
```

} }

- TestNG marks test as: FAILED, If a test method exceeds the defined timeout. Test execution is **immediately terminated.**
- **Use cases:** Page Load Validation, API Response Time, Prevent tests from hanging

---

Built - in reports in TestNG:

- Emailable Report (emailable-report.html): TestNG automatically generates a set of **default reports** after execution.
  - A **simple HTML report** generated by TestNG for quick viewing.
  - It contains Total test count, Passed / Failed / Skipped tests, Execution summary, Method-level results
- Index Report (index.html): A **detailed interactive report** with navigation.
  - It contains Suite-level summary, Test-level breakdown, Groups information, Execution timeline, Reporter output
- TestNG Results XML Report (testng-results.xml):
  - It contains Test status (pass/fail/skip), Execution time, Parameters, Exception details
- JUnit Report (junitreports/ folder):
  - JUnit Report (junitreports/ folder). Easily integrated with reporting dashboards
- Reporter Output (reporter-output.html):
  - Captures logs written using: Reporter.log("Message");
  - It contains custom logs from test execution.
- Failed Tests Report (testng-failed.xml):
  - Automatically generated XML file containing **only failed tests.**
  - After execution: Run only failed tests again

Custom Reports in TestNG:

- TestNG's default reports are **basic**, so in real projects we use external tools to generate **rich, professional reports**.
- Two popular tools:
  - Extent reports
    - Extent Reports focuses on **detailed and structured reporting**.
    - It provides Step-by-step logs, Screenshots on failure, Pie charts (pass/fail), Test execution timeline
  - Allure
    - Allure provides **more advanced visualization and analysis features** compared to basic reports.

        - It provides Interactive UI, Filters (by status, feature, severity), Test history & trends, Step-level execution view, Attachments (screenshots, logs, videos)

Handling Expected Exceptions in TestNG:

- It can be handled using the expectedExceptions attribute in the @Test annotation. Ex. @Test(expectedExceptions = ExceptionType.class)
- TestNG allows you to **expect a specific exception** in a test. If that exception occurs → test PASSES. If that exception occurs → test PASSES.

```
public class ExceptionTest {

   @Test(expectedExceptions = ArithmeticException.class)

   public void divideByZeroTest() {

       int result = 10 / 0; // throws ArithmeticException

   }

}
```

- This is mainly used to **validate error handling logic**.
- Real scenario: User tries login without password. Application throws exception like: IllegalArgumentException: Password required. Test should PASS because system handled error correctly.

Validating Exception Messages:

- `expectedExceptionsMessageRegExp` attribute is used to validate specific exception messages.
- Sometimes just checking exception type is not enough. We also want to verify **exact error message**.

```
Ex. @Test(

expectedExceptions = Exception.class,

expectedExceptionsMessageRegExp = "message"

)
```

- Test will PASS only if: Exception type matches and Exception message matches.

Retrying Failed Tests in TestNG:

- TestNG provides two key interfaces: IRetryAnalyzer → defines **retry logic,** IAnnotationTransformer → applies retry logic dynamically to tests. Together, they allow automatic re-execution of failed tests.
- IRetryAnalyzer : It defines how many times a test should be retried.
  - It allows TestNG to **automatically re-execute failed tests**.
  - To use retry functionality: Implement IRetryAnalyzer interface and Override retry() method. retry() is called automatically when a test fails.
  - We define a **maximum retry count** to avoid infinite execution.

```
public class RetryAnalyzer implements IRetryAnalyzer {

int count = 0;

int maxRetry = 2;

@Override

public boolean retry(ITestResult result) {

if (count < maxRetry) {

count++;

return true; // retry test

}

return false; // stop retry

}    }
```

Applying Retry Logic to Test:

```
public class LoginTest {

@Test(retryAnalyzer = RetryAnalyzer.class)

public void verifyLogin() {

System.out.println("Executing Login Test");

// simulate failure

assert false;

}    }
```

- IAnnotationTransformer: Instead of adding retryAnalyzer in every test: IAnnotationTransformer automatically attaches retry logic to all tests.

  public class RetryTransformer implements IAnnotationTransformer {

  @Override

  public void transform(ITestAnnotation annotation, Class testClass,

  Constructor testConstructor, Method testMethod) {

  annotation.setRetryAnalyzer(RetryAnalyzer.class);

  } }

  testng.xml:

  <listeners>

      <listener class-name="listeners.RetryTransformer"/>

  </listeners>

- Use cases: Handling Flaky Tests, UI timing issues, Third-Party Dependency Failure

---

TestNG test run with Maven:

- TestNG test cases can be executed using Maven by setting up the **Maven Surefire Plugin** inside the pom.xml configuration file.
- This setup enables TestNG test suites to run automatically whenever the Maven build is executed.
- When run : mvn clean test
  - Maven will: Compile code, Execute TestNG tests via Surefire, Generate reports
- This is very important for: CI/CD pipelines (Jenkins, GitHub Actions), Automated regression testing, Continuous feedback

---

TestNG with cucumber:

- Cucumber tests can be executed through TestNG by using a TestNG runner class that connects Cucumber scenarios with the TestNG framework.
- Test scenarios are written in Gherkin format (feature files), and each step is linked to Java methods (step definitions) that are executed via TestNG.

Feature file:

```
Feature: Login Feature

Scenario: Valid Login

Given User is on login page

When User enters valid credentials

Then User should be logged in
```

Step Definition file:

```
import io.cucumber.java.en.*;

public class LoginSteps {

    @Given("User is on login page")

    public void openLoginPage() {

        System.out.println("Opening Automation Test Store");

    }

    @When("User enters valid credentials")

    public void enterCredentials() {

        System.out.println("Entering username and password");

    }

    @Then("User should be logged in")

    public void validateLogin() {

        System.out.println("Login successful");

    }

}
```

- Reports can be generated by configuring plugins such as Cucumber Reports or Extent Reports in the Cucumber runner class.

```
@CucumberOptions(
plugin = {
"pretty",
"html:target/cucumber-report.html",
```

```
"json:target/cucumber.json"
}
)
```

- TestNG can be used alongside Cucumber, allowing you to combine BDD scenarios with TestNG's execution and configuration capabilities.

Handling Test Failures and Debugging in TestNG:

- Troubleshoot failed tests: Enable logging, Add logs in your test to understand execution flow.

```
@Test
public void loginTest() {

Reporter.log("Opening application");
Reporter.log("Entering username");
Reporter.log("Clicking login button");
// failure happens here
}
```

  - Analyze Stack Trace: Stack trace shows **exact failure reason**
    Example error: ElementNotInteractableException:
    Element is not clickable at point
- Debug TestNG tests in IDE:
  - Run test in **debug mode instead of normal run.**
  - Pause execution at a specific line.
  - Check values during execution

Thread:

- A thread is the smallest unit of execution inside a program. It represents a single flow of tasks that can run independently.
- Each thread is used to execute a test case (method or class). When multiple threads are used, multiple tests can run at the same time.

How `thread-count` Works:

- **Single Thread:**
  - If you don't specify `thread-count`, TestNG uses only one thread, so all tests run sequentially (one after another).
- **Multiple Threads:**
  - When you define `thread-count`, TestNG creates multiple threads and executes tests simultaneously.

Usage in TestNG (testng.xml):

- The thread-count attribute is defined inside testng.xml and controls how many threads (parallel executions) should run at the same time.
- It works together with the parallel attribute, which defines **what level you want to parallelize**.

Influence of Thread Count in Parallel Execution:

- Execution Time Optimization: When you increase thread-count, TestNG runs **multiple tests at the same time (parallel)** instead of one-by-one.
- System Resource Utilization: Each thread uses CPU and uses memory. More threads = More memory.
- Ensuring Test Data Isolation: Since tests run simultaneously: They should NOT depend on shared data/resources.

Configuring the correct thread count:

- Small Test suits: If you have fewer tests, No need for too many threads. Just 2-4 thread-count is enough.
- Large Test suits: For hundreds of tests, you can increase the thread count to 8 to 10.
- System resource availability: Consider CPU and memory while setting thread-count. Always check: CPU cores, RAM etc…

---

alwaysRun = true in TestNG:

- It ensures that a method **will execute no matter what**, even if: A dependent method fails, A group is skipped, Some configuration fails.
- It is commonly used with: @Test, @BeforeMethod, @AfterMethod, etc.

---

invocationCount in TestNG:

- It allows you to specify how many times a test method should be executed.
- Sometimes a test passes once but may fail randomly. Running it multiple times helps detect **flaky behavior.**
- Used to verify if the application behaves consistently under repeated execution.

```
Example: @Test(invocationCount = 10, threadPoolSize = 3)
public void loginLoadTest() {
    System.out.println("Executing login in parallel");
}
//10 executions and 3 threads running in parallel.
```